



EMI305 · PRODUCCIÓN DE SOFTWARE · SISTEMAS CIBERFÍSICOS

Aplicación de transformaciones MDD4CPS al caso SVIF

Actividad Semana 4 — proceso MDD4CPS aplicado sobre el caso SVIF, siguiendo la cadena CIM → PIM → PSM → Code.

Magíster en Ingeniería Informática · UFRO

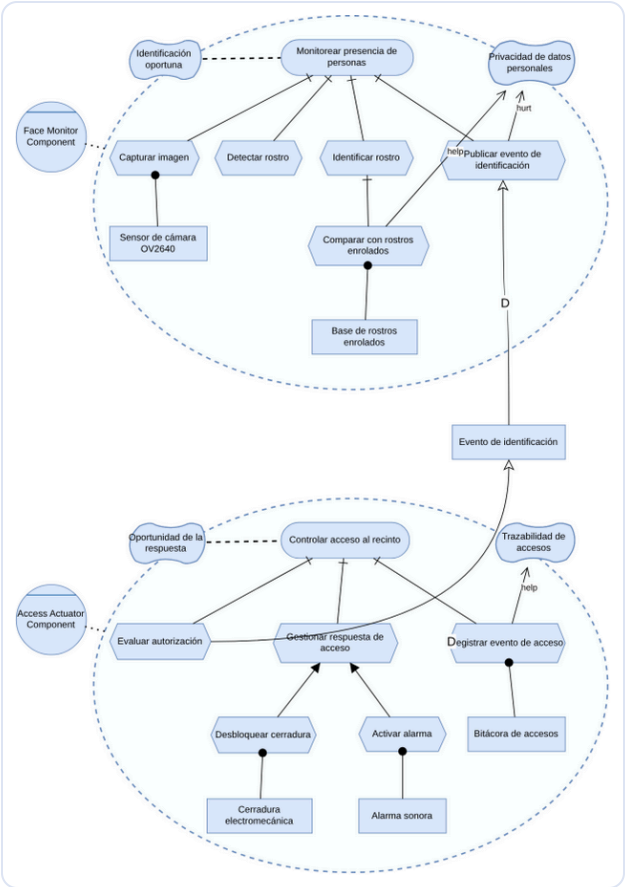
EMI305 · Producción de Software · 2026



INSTITUCIÓN ACREDITADA
6 AÑOS
EN TODAS LAS ÁREAS
• GESTIÓN INSTITUCIONAL • DOCENCIA DE PREGRADO
• DOCENCIA DE POSTGRADO • INVESTIGACIÓN
• VINCULACIÓN CON EL MEDIO

SVIF y su modelo CIM

SVIF (Sistema de Videovigilancia con Identificación Facial) — dos nodos: **Face Monitor** (ESP32-CAM) y **Access Actuator** (cerradura v alarma). Vista híbrida SD/SR del CIM en iStar 2.0:



La vista muestra ambos agentes con sus metas, tareas, softgoals, refinamiento OR (desbloquear v alarmar) y la dependencia «Evento de identificación». Detalle en *cim-istar-svif.drawio* (página 2).

Transformación CIM → PIM

Cada constructo del CIM se tradujo a un constructo del DSL, manteniendo la trazabilidad mediante id_cim_parent:

CIM (ISTAR)	PIM (DSL)	CANTIDAD
Agent	CPCComponent	2
Goal	OnIntervalAction	2
Task	OnDemandAction	10
Resource	SW / HW Resource	2 SW · 3 HW
Dependency	MessageSender + Receiver	1 + 1
Refinamientos AND/OR	Operadores AND/OR	2 + 1
Quality	qualification/contribution_array	4 propagados

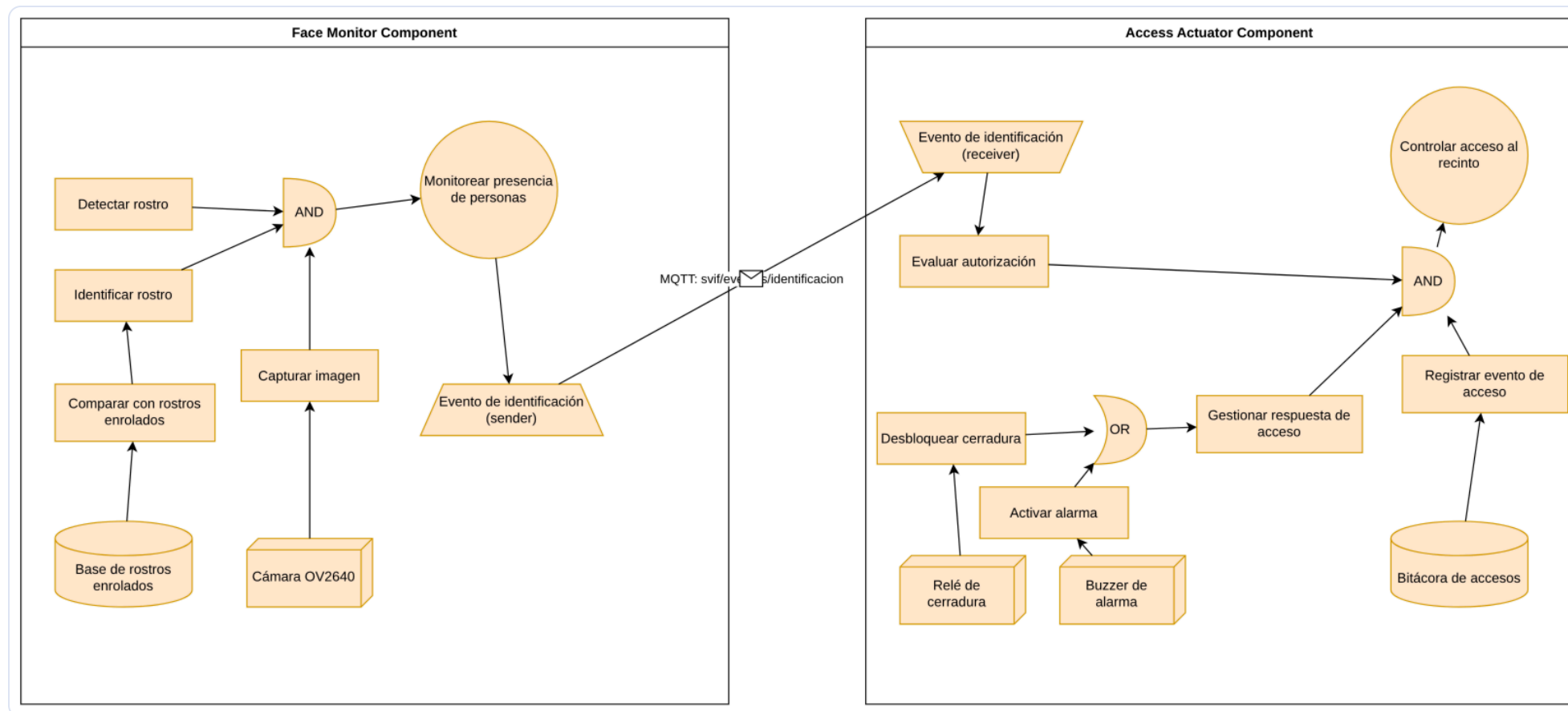
Decisiones del diseñador en esta etapa:

- PERÍODOS: 500 MS (MONITOR) · 250 MS (ACTUADOR)
- PARÁMETROS DE ENTRADA/SALIDA DE LAS TAREAS
- ESTRUCTURA DEL DEPENDUM: TIMESTAMP · PERSON_ID · PERSON_NAME · CONFIDENCE · AUTHORIZED

No guardamos imágenes, sino una firma a partir del rostro: el dependum transmite sólo identidad y metadatos.

Materialización PIM → PSM sobre ESP32

Decisiones de plataforma: **ESP32** (Arduino core), comunicación **MQTT**, tipado concreto en C++. Vista del PIM en el DSL para CPS:



El PIM muestra los dos *CComponent* (Face Monitor / Access Actuator), sus *OnIntervalAction* / *OnDemandAction*, *SWResource* / *HWResource*, y los *MessageSender* + *MessageReceiver* del dependum. Detalle en *pim-dsl-svif.drawio*.

Completado manual y verificación

Lo que se completó manualmente:

- Credenciales del broker MQTT (`secrets.h`).
- Umbral de confianza para la identificación.
- Política de autorización.
- Modo de simulación (`SIMULATION_MODE`) para probar sin hardware.

Verificación de punta a punta contra un broker Mosquitto local:

DETECCIÓN

IDENTIFICACIÓN LOCAL

PUBLICACIÓN MQTT

EVALUACIÓN

DESBLOQUEO | ALARMA

BITÁCORA

14 eventos registrados: 10 desbloques + 4 alarmas.

HALLAZGO

La rama de alarma no se ejercitaba

Con la regla sintética inicial, todos los eventos resultaban en desbloqueo.

Se ajustó la política de autorización para que la rama de alarma se active cuando `confidence < umbral` o `authorized = false`.

Detalle en `evidencia-de-pruebas.md`.

Aplicación de la herramienta aomdd4cps

Como validación cruzada, se ejecutó la herramienta aomdd4cps (Flask + SaxonC/XSLT en Docker) sobre el mismo CIM. Primero se reprodujo el caso **greenhouse** del repositorio: PIM y PSM idénticos a los de referencia → el entorno es fiel. Luego, el mismo pipeline sobre SVIF:

CIM-ISTAR-SVIF.DRAWIO → CIM → PIM (XSLT) → PIM → PSM (XSLT) → PSM_TO_CODE.PY

FASE 1 · CIM → PIM

🟡 Fiel, con una fuga

2 cps_component, 2 operational_goal, 10 action, recursos y comunicación correctos. **Descarta los operadores AND/OR.**

FASE 2 · PIM → PSM

✅ Completo

2 cpc, 10 function, 2 thread con período (500/250 ms), commThread + listenerThread.

FASE 3 · PSM → CODE

❌ No compila

622 líneas, pero apunta a **Arduino MKR 1010** (no ESP32), contiene char[32] x; inválido y **22 stubs** de lógica.

Modelos, código generado y scripts reproducibles en *semana-4/comparativa-app/*.

Análisis del código generado

Para estimar el código generado por la transformación, se contaron las líneas producidas automáticamente (.ino + comm_utils.h) y se descontaron las secciones completadas manualmente en la fase Code.

CPC	LÍNEAS TOTALES	PERSONALIZADAS	% GENERADO
Face Monitor Component	310	~70	~77 %
Access Actuator Component	315	~65	~79 %
Total	625	~135	~78 %

El ~78 % coincide con lo reportado por Navarro et al. (2025) para el caso del invernadero.

APORTE DE MDD4CPS

Qué entrega bien

- Trazabilidad cim-* → id_cim_parent: cada función queda enlazada a su tarea CIM.
- Decisiones postergadas al nivel correcto (plataforma sólo aparece en el PSM).
- Andamiaje homogéneo de hilos y comunicación.

QUÉ FALTA

Limitaciones verificadas

- El DSL no expresa restricciones temporales duras.
- Los softgoals quedan sólo como comentarios (no verificables).
- La herramienta aomdd4cps descarta los operadores AND/OR.
- El emisor de código apunta a otra placa y produce sintaxis inválida.

Reflexiones finales

Aprendizajes:

- Correr la herramienta oficial sobre el mismo CIM reveló dos limitaciones que la narrativa inicial no detectaba: el descarte de AND/OR y el código que no compila.
- La trazabilidad `cim-*` hace navegable la pregunta «qué softgoal sobrevive en qué función».
- El DSL no verifica restricciones temporales: la simulación es indispensable.
- El ~22 % manual no es ruido: son credenciales, política de autorización y detalles que el modelo no deduce.

Para una próxima iteración:

- Usar la herramienta para el andamiaje y el agente para la fase Code.
- Definir la política de autorización antes de la primera ejecución.
- Detallar los recursos HW en el PIM (pines, polaridad).
- Documentar el mapeo softgoals → criterios de aceptación desde el CIM.

Comparativa final: agente y herramienta

DIMENSIÓN	HERRAMIENTA A0MDD4CPS	AGENTE
Determinismo y reproducibilidad	✓ (XSLT puro)	—
Homogeneidad del andamiaje	✓	—
Proceso auditable por reglas	✓	—
Operadores AND/OR preservados	✗	✓
Código que compila	✗	✓
Plataforma del caso (ESP32)	✗ (MKR 1010)	✓
Fase Code cubierta	✗ (22 stubs)	✓
Verificado punta a punta	✗	✓

Lectura. Las dos vías cubren aspectos distintos. La herramienta es fuerte en rigor y reproducibilidad del andamiaje; el agente es fuerte en corrección, plataforma adecuada, semántica del OR y cobertura de la fase Code. No son intercambiables: cada una aporta lo que la otra no entrega en su estado actual.

Detalle en semana-4/comparativa-agente-vs-app.md · evidencia en semana-4/comparativa-app/ (4 modelos XML + código + scripts).

Entregables de la actividad

ENTREGABLE	ESTADO	ARCHIVO
Modelo CIM (iStar 2.0)	✓	cim-istar-svif.drawio
Modelo PIM (DSL)	✓	pim-dsl-svif.drawio
Código fuente (ESP32)	✓	codigo/
Validación con la herramienta	✓	comparativa-app/
Comparativa agente vs. herramienta	✓	comparativa-agente-vs-app.md
Presentación	✓	este PDF
Video (5–7 min)	☐	grabar (guion listo)
Encuesta	☐	forms.gle/VPaL9qx7mhKttJgf8

Bézivin, J. (2005). *On the unification power of models. Software & Systems Modeling*, 4(2), 171–188.

Dalpiaz, F., Franch, X., & Horkoff, J. (2016). *iStar 2.0 language guide. arXiv:1605.07767.*

Navarro, C., Devia, L., Labra Gayo, J. E., & Cares, C. (2025). *An agent-oriented model-driven development process for cyber-physical systems. CIBSE 2025* (pp. 150–164). SBC.
<https://doi.org/10.5753/cibse.2025.35298>

Repositorio: github.com/mdd4cps/aomdd4cps (CC BY-NC 4.0)